

NOCA – A Notification-Oriented Computer Architecture

R. R. Linhares, J. M. Simao and P. C. Stadzisz

Abstract—Current software development processes lack techniques for the productive and quality design of software that makes efficient use of the parallel execution capabilities provided by the hardware of modern computing systems. In this context, the Notification Oriented Paradigm (NOP) has been recently developed aiming at a new organization of software logic based on notifications among causal-logical entities. NOP allows exploring the parallelization and/or distribution in a simpler and more efficient way than more commonly used programming paradigms. However, the execution dynamics under the NOP, based on notifications, is not efficiently performed by the hardware of current computing systems. This paper presents a new computer architecture, named NOCA, which is suitable for execution of software developed according to the NOP computing model. NOCA was designed in accordance with the principles of generality and scalability, which allow it to execute NOP software at any level of complexity by fetching it from a program memory. The developed architecture is organized as a fine grain multiprocessor that hierarchically executes instructions through sets of specialized processor cores. Preliminary experiments performed on this architecture show that NOCA presents improvements in terms of performance comparative evaluations.

Keywords— Notification-Oriented Paradigm, Computer Architecture, Parallel Computer Architecture.

I. INTRODUÇÃO

A EVOLUÇÃO recente das arquiteturas de computadores tem ocorrido no sentido de se projetar e construir processadores com múltiplos núcleos [1][2]. No entanto, melhorias de desempenho são dependentes de um uso eficiente dos múltiplos núcleos de processamento disponíveis nestas novas plataformas. Técnicas de programação que são estado-da-arte, como o Paradigma Orientado a Objetos (POO) e os Sistemas Baseados em Regras (SBR) são sujeitos a limitações intrínsecas relacionadas a acoplamento e redundâncias, as quais dificultam o uso dos recursos de paralelização e distribuição disponíveis nas arquiteturas de múltiplos núcleos. Como consequência, há uma demanda crescente por técnicas e ferramentas que sejam adequadas para o desenvolvimento de *software* paralelo.

Neste contexto, propôs-se recentemente o Paradigma Orientado a Notificações (PON). O PON é baseado em uma teoria de controle holônico [3] e propõe eliminar algumas das deficiências encontradas em paradigmas atuais no que diz respeito a avaliações causais acopladas e desnecessárias. O PON é caracterizado por um modelo lógico organizado na

forma de entidades computacionais lógico-causais e entidades factuais, cuja inferência ocorre por meio de notificações. As entidades lógico-causais atuam como regras, cuja ativação ocorre reativamente em resposta a mudanças de estado das entidades factuais. Por meio de notificações precisas entre os fatos e as regras, elimina-se a necessidade de mecanismos adicionais de inferência para efetuar as operações de *matching* entre eles [4].

Algumas abordagens já utilizadas para se desenvolver sistemas de acordo com o PON incluem: a prototipação de um *framework* C++ para permitir a implementação de *software* PON baseado em abstrações do POO como classes e objetos [5]; um co-processador PON (CoPON) baseado em *hardware* reconfigurável para uma determinada aplicação, permitindo a execução híbrida do processo de inferência do PON em colaboração com um núcleo von Neumann [6]; e a definição de modelos de circuitos digitais que permitem a implementação, em total conformidade com os princípios do PON, de uma determinada aplicação em *hardware* reconfigurável [7].

No entanto, todas estas abordagens apresentam alguma deficiência na execução de *software* orientado a notificações. Por exemplo, o *framework* C++ depende de estruturas de dados baseadas no Paradigma Imperativo (PI) e de buscas sobre estas estruturas para implementar a dinâmica de notificações do PON; o *overhead* resultante pode ser minimizado por meio do uso de um compilador PON para simplificar tais estruturas, o qual está em desenvolvimento. Do ponto de vista de *hardware*, por sua vez, as iniciativas citadas mapeiam uma aplicação PON ou parte dela em circuitos digitais especializados. Estas abordagens carecem, portanto, de generalidade e escalabilidade no sentido de que qualquer alteração na lógica da aplicação requer reconfiguração de todo o circuito, além do que o seu tamanho e complexidade estão limitados à capacidade do dispositivo de lógica reconfigurável.

Este artigo apresenta uma nova abordagem para execução de *software* PON, na forma de uma arquitetura de computador denominada NOCA (*Notification-Oriented Computer Architecture*). Esta arquitetura objetiva implementar a dinâmica de notificações de PON de forma mais próxima possível ao seu modelo teórico, permitindo que se usufrua de características tais como a relativa facilidade de distribuição e paralelização providas pelo PON. Além disso, a NOCA objetiva generalidade, de tal maneira que seja possível alternar aplicações PON diferentes somente pela substituição de *software* em memória de programa, de forma similar ao que ocorre em computadores tradicionais.

A NOCA é apresentada por meio da descrição de seus modelos estrutural e dinâmico. Além disso, discorre-se sobre

R. R. Linhares, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, linhares@utfpr.edu.br

J. M. Simao, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, jeansimao@utfpr.edu.br

P. C. Stadzisz, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, stadzisz@utfpr.edu.br

resultados de experimentos preliminares, os quais comparam o desempenho e complexidade de aplicações desenvolvidas segundo o PON e executando na NOCA, versus tais aplicações desenvolvidas segundo o PI e executando em um núcleo von Neumann e também implementadas por meio de síntese de *hardware* em FPGA.

As próximas seções deste artigo estão organizadas da seguinte maneira: as seções 2 e 3 apresentam a teoria a respeito do PON e uma visão geral sobre arquiteturas de computadores paralelas. A seção 4 apresenta a arquitetura proposta. A seção 5 apresenta uma discussão sobre os experimentos preliminares. Por fim, a seção 6 apresenta conclusões e perspectivas de trabalhos futuros.

II. PARADIGMA ORIENTADO A NOTIFICAÇÕES (PON)

O PON foi proposto como um novo paradigma de desenvolvimento de *software*, de forma a oferecer um novo modelo lógico que contribuisse para aumentar a produtividade e a qualidade no processo de desenvolvimento. Adicionalmente, o PON viabiliza melhorias no desempenho do *software* e uma maior facilidade de construção de sistemas complexos, especialmente paralelos ou distribuídos. O modelo lógico do PON permite reduzir ou mesmo eliminar alguns dos problemas relativos a paradigmas clássicos tais como o Imperativo (PI) ou o Declarativo (PD) (particularmente os Sistemas Baseados em Regras, ou SBR). Como exemplos de tais problemas pode-se citar o forte acoplamento entre entidades no cálculo lógico-causal e também as redundâncias estruturais e temporais [8].

Software desenvolvido de acordo com o PON compreende um conjunto de entidades lógico-causais e de entidades factuais definidas pelo projetista. Estas entidades colaboram por meio de notificações entre elas, definindo um processo de inferência inovador que é distinto daqueles utilizados no PI e nos SBRs. Ainda, a estrutura das entidades lógico-causais envolve expressões imperativas (i. e. lógicas e funcionais) que são análogas às expressões utilizadas no PI [9].

O modelo lógico do PON pode ser descrito por meio de um diagrama de classes, conforme mostrado na Fig. 1. Os elementos factuais são modelados por meio da classe *FBE* (*Fact Base Element*, ou Elemento da Base de Fatos). Cada entidade é definida por um *FBE* e agrega um conjunto de atributos (i. e. variáveis de estado) e métodos (i. e. funções ou operações) descritos pelas classes *Attribute* e *Method*, respectivamente. Os elementos lógico-causais, por sua vez, são modelados por meio da classe *Rule*. Cada entidade definida como uma *Rule* representa uma regra lógica que agrega uma condição e uma ação descritas pelas classes *Condition* e *Action*, respectivamente. A condição de uma regra define o cálculo lógico sobre um conjunto de premissas cujo resultado determina se a regra é ativada ou não. Cada premissa é definida por um objeto da classe *Premise* e compreende uma expressão relacional sobre um ou dois atributos (i. e. uma entidade *FBE*). A ação de uma regra é responsável por disparar um conjunto de instigações, descritas pela classe *Instigation*, representando o que deve ser processado quando uma regra é ativada. *Instigations* ativam métodos a serem

executados, descritos pela classe *Method*, e que contêm a lógica funcional que atua sobre os atributos de um *FBE*.

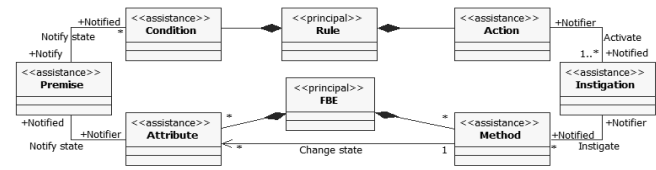


Figura 1. Modelo lógico do PON [4].

Para cada alteração no estado de um atributo de *FBE*, suas premissas dependentes são notificadas por ele e executam suas operações relacionais, atualizando seu estado. Ainda, caso o estado de uma premissa mude, suas condições dependentes são notificadas por ela e executam seus cálculos lógicos, assim atualizando seu estado. Se o resultado do cálculo lógico de uma condição é *true*, a regra respectiva é aprovada e pode ser ativada, resultando na execução de sua ação.

A Fig. 2 apresenta um exemplo de regra que é parte de um simulador de controle de tráfego utilizando semáforos. A *Condition* desta *Rule* efetua a decisão de decrementar o tempo em que um semáforo permanece no estado “fechado”, dependendo do número de veículos aguardando o semáforo mudar para o estado “aberto”. Esta *Condition* avalia três *Premises* que efetuem a seguinte avaliação sobre os *FBEs*, i. e., os semáforos: a) o número de veículos enfileirados é maior ou igual a 10? b) o semáforo está “fechado”? e c) o semáforo está “fechado” por 30 segundos ou mais?

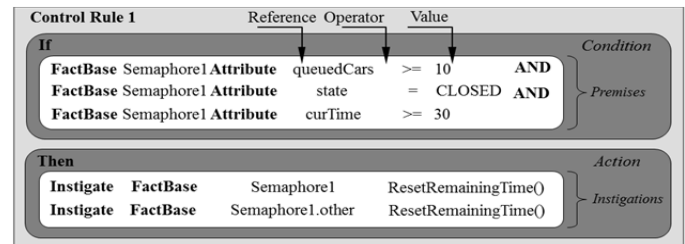


Figura 2. Exemplo de regra PON.

Neste exemplo, a *Action* contém duas *Instigations* para: a) reinicializar o tempo restante para o *Semaphore1* alterar seu estado, o que indiretamente causa a alteração do seu estado para “aberto” por meio da avaliação de outra *Rule*; e b) reinicializar o tempo restante para que o outro semáforo, que pertence ao mesmo cruzamento, mude o seu estado, o que indiretamente causa a alteração do seu estado para “fechado” por meio de avaliação de outra *Rule*, dado que o estado deste semáforo é o oposto do estado do *Semaphore1* por definição. Efetivamente, o que uma *Instigation* faz é instigar um ou mais *Methods* responsáveis por efetuar os serviços ou operações definidos em um *FBE*.

A Fig. 3 apresenta um exemplo de *software* PON composto de duas entidades lógico-causais (*Rules*) e duas entidades factuais (*FBEs*). Esta figura ilustra a composição da cadeia de

notificações que integra as entidades de acordo com o modelo lógico do PON. Ela também mostra a estrutura que habilita a dinâmica de execução de acordo com o paradigma.

As linhas em negrito da Fig. 3 representam o fluxo de notificações para a ativação das regras, iniciando com a mudança de um *Attribute* e propagando para os elementos notificados, com a aprovação de uma ou mais regras como resultado. As linhas mais claras, por sua vez, representam o fluxo de notificações para a execução das regras, resultando na ativação dos *Methods* que atualizam valores de *Attributes* conforme indicado pelas linhas tracejadas. Pode-se perceber que a essência da computação no PON é organizada e distribuída entre entidades autônomas e reativas, as quais

colaboram por meio de notificações. Esta estrutura define o mecanismo de notificações e o fluxo de execução da aplicação como consequência.

O PON leva a uma nova forma de se projetar *software*, na qual o fluxo de execução está distribuído entre um conjunto de entidades fracamente acopladas (i. e. regras). Diferentes fluxos de execução sequenciais ou paralelos podem existir devido à sequência lógica das regras. Portanto, a expressão do paralelismo é intrínseca ao modelo lógico do PON, tornando-o uma alternativa para o projeto de *software* paralelo e, potencialmente, até mesmo mais vantajoso do que outras técnicas existentes.

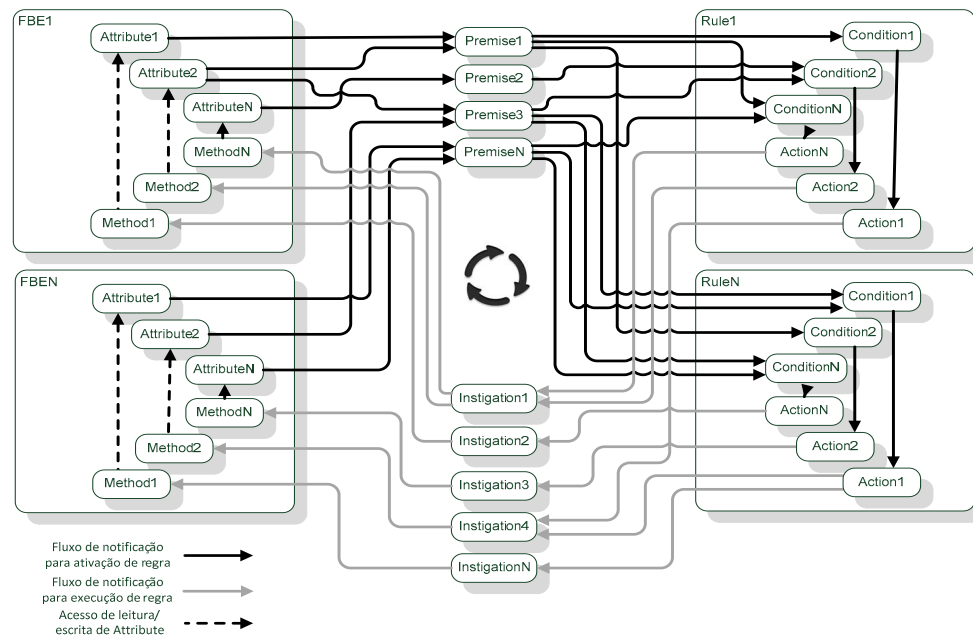


Figura 3. Exemplo de cadeia de notificações PON [10].

III. ARQUITETURAS DE COMPUTADORES PARALELOS

A facilidade com a qual *software* com paralelismo pode ser desenvolvido e executado representa uma das características implícitas do PON que deve ser levada em consideração para a proposição da NOCA. Esta seção revisa brevemente a história dos modelos de arquiteturas de computador paralelas, em particular os mais relevantes, para permitir o posicionamento da NOCA em relação a estes modelos,

Ao longo da história recente da computação, muitos esforços foram realizados para desenvolver e melhorar arquiteturas de computadores, objetivando executar *software* com algum nível de paralelismo. Dois modelos de computação foram particularmente mais empregados para projetar arquiteturas de computador paralelas:

- *Modelo de von Neumann*, segundo o qual um programa é armazenado em memória e tem sua próxima instrução

buscada pelo processador no endereço de memória indicado por um contador de programa. O endereço apontado pelo contador de programa é incrementado sequencialmente ou atualizado com o resultado de instruções de salto (*branches*) utilizadas para redirecionar o fluxo de controle do programa.

- *Modelo de fluxo de dados*, segundo o qual as operações (instruções) do programa têm a sua execução habilitada e efetuada à medida que os dados (operandos) dos quais elas dependem se tornam disponíveis [11].

É possível obter paralelismo em vários níveis de abstração no modelo de von Neumann, mediante o uso de várias técnicas. Por exemplo, o Paralelismo em Nível de Instrução (*Instruction Level Parallelism*, ou ILP) pode ser obtido por meio de técnicas tais como *pipelining* [12]. Em um nível mais alto de abstração, Paralelismo em Nível de Thread (*Thread Level Parallelism*, ou TLP) pode ser obtido por meio do

particionamento do *software* em diferentes fluxos de controle, cuja execução é suportada por recursos arquiteturais tais como *multithreading* [13], a disponibilização de múltiplos núcleos de execução no mesmo *chip* (CMP, ou *Chip Multiprocessor*), nas chamadas arquiteturas *multicore*, ou mesmo a implementação de sistemas híbridos, que combinam sistemas de memória compartilhada (tipicamente *multicore*) com sistemas de memória distribuída (multiprocessadores) [14]. Embora esta seja a abordagem mais comum atualmente para o projeto de microprocessadores, ela apresenta alguns desafios adicionais, tais como: o gerenciamento de concorrência entre acessos simultâneos à memória compartilhada, efetuados pelos diferentes núcleos; assegurar a coerência na memória *cache* em implementações nas quais cada núcleo possui uma *cache* separada para seu uso individual.

O foco no desenvolvimento de computadores compatíveis com o modelo de von Neumann ao longo dos anos [15] favoreceu a propagação de linguagens e modelos de programação compatíveis. O modelo de programação de um computador von Neumann é fundamentalmente baseado em variáveis para armazenamento, instruções de controle como uma forma de gerenciar o fluxo de execução, e instruções de atribuição para implementar leituras e escritas de memória [16]; Este modelo é a base tecnológica para o PI e, conseqüentemente, para as linguagens de programação procedimentais e orientadas a objeto mais populares, tais como C/C++ e Java. Portanto, a adaptação do desenvolvedor a novos *frameworks*, ambientes de desenvolvimento ou linguagens de programação imperativas que sejam compatíveis com o modelo de von Neumann requer relativamente pouco esforço, o que é uma vantagem considerável sobre o uso de outros modelos de programação ou computação menos populares.

Como desvantagem, independentemente da categoria ou nível de paralelismo, o modelo de von Neumann apresenta uma característica conhecida como “gargalo de von Neumann” que consiste em frequentemente buscar instruções da memória fora do núcleo de execução, cuja latência e largura de bandas são tipicamente limitadas [15]. Estas limitações aumentaram nos últimos anos devido ao efeito conhecido como *memory wall*, que é a discrepância crescente entre o desempenho de execução dos processadores e o desempenho de acesso à memória [17]. De acordo com [15], esta discrepância alcançou um fator de 1000 em 2007 e continua a crescer. De fato, o modelo de von Neumann não foi projetado para a execução de *software* paralelo. Mesmo que a execução dos programas possa ser paralelizada até certo ponto, esta paralelização impõe formas de comunicação e sincronização entre as *threads* envolvidas, com o conseqüente *overhead* de execução [18]. Adicionalmente, o modelo de execução restringe a expressão da lógica paralela pelo programador.

No contexto do modelo de fluxo de dados, por sua vez, a estrutura do programa em si favorece a execução paralela de instruções individuais. Ou seja, as instruções estão aptas a serem (paralelamente) executadas tão logo seus operandos estejam disponíveis. Do ponto de vista tecnológico, muitas

técnicas arquiteturais foram desenvolvidas para ajudar a melhorar algumas características da execução de fluxo de dados. Um exemplo é o uso de *Explicit Token Stores (ETS)* para otimizar a busca e despacho dos dados disponíveis para as unidades de execução apropriadas [19]. Outro exemplo consiste nas *I-Structures*, que são blocos de memória que definem uma semântica de sincronização específica de forma a gerar uma notificação para uma instrução de fluxo de dados sobre a disponibilidade dos dados dos quais ela depende [20].

O modelo de fluxo de dados é teoricamente mais adequado para a computação paralela do que o modelo de von Neumann porque a sua execução é intrinsecamente paralela e porque ele apresenta abstrações mais adequadas para o desenvolvimento de programas concorrentes [21]. Contudo, algumas questões relativas à necessidade de se efetuar um processo de busca para o *matching* de operandos disponíveis, bem como o impacto da granularidade fina de execução no *pipeline* e no número de acessos aos níveis de memória inferiores, afetam o desempenho destas arquiteturas e dificultaram o seu uso. Há também algumas desvantagens inerentes ao modelo de programação de fluxo de dados, tais como a ausência de variáveis, que tornam esta programação mais similar a um modelo de programação funcional, o qual pode ser difícil de utilizar e ineficiente para muitas aplicações [22].

IV. DESCRIÇÃO DA NOCA

Esta seção apresenta a arquitetura de computador orientada a notificações proposta, chamada de NOCA, a qual é uma alternativa às arquiteturas tradicionais baseadas em modelos tais como von Neumann ou fluxo de dados. A NOCA suporta a execução de aplicações de *software* desenvolvidas segundo o PON, conferindo-lhes flexibilidade por viabilizar que estas sejam alteráveis ou substituíveis em memória.

A. Modelo lógico da NOCA

O desenvolvimento da NOCA foi baseado em um conjunto de requisitos genéricos relacionados a características do PON e ao seu modelo de execução, conforme listado abaixo.

- A NOCA deve ser capaz de executar *software* composto unicamente de elementos do PON e, opcionalmente, também de funções sequenciais de acordo com o modelo de von Neumann.
- A NOCA deve ser genérica, no sentido de que qualquer alteração na aplicação PON sendo executada dependa somente de alterações de *software*, portanto não requerendo qualquer reconfiguração de *hardware*.
- A NOCA deve definir uma arquitetura de conjunto de instruções (*Instruction Set Architecture*, ou ISA) que implemente as funcionalidades dos elementos da cadeia de notificações do PON.
- A NOCA deve definir unidades de processamento que sejam capazes de executar as instruções da ISA e o fluxo de notificações de forma paralela.
- A NOCA deve ser capaz de executar uma aplicação PON mesmo que esta seja composta de mais elementos notificantes do que o número de unidades de processamento disponíveis para sua execução. Isto

viabiliza a escalabilidade, no sentido de que o tamanho de uma aplicação PON a ser executada é limitado somente pela quantidade de memória disponível para armazenamento do respectivo *software*.

Com base nestes requisitos, o modelo lógico da NOCA foi projetado conforme mostrado na Fig. 4. O metamodelo de notificações do PON está posicionado no lado esquerdo e a forma como este metamodelo mapeia para a ISA da NOCA é mostrado ao centro. Alguns dos elementos do metamodelo (*Rule*, *Action* e *Instigation*) não são mapeados para qualquer instrução específica, uma vez que não executam qualquer operação de cálculo ou lógico-causal, porém atuam unicamente como agregadores ou roteadores de notificações em um *software* PON.

Ao seu turno, o pacote no lado direito da Fig. 4 mostra a ISA mapeada para alguns processos de *hardware*, responsáveis por executar as respectivas instruções. Estes processos são agrupados em conjuntos, cada um deles representando um grupo de processadores que envia e recebe notificações por meio do mesmo canal. Adicionalmente, são definidos: um processo específico para detecção de alterações em *Attributes*, associado com o repositório correspondente; um processo responsável por escalonar ações e resolver conflitos entre *Rules* (*Scheduler/Conflict solver*, ou S/CS); e um núcleo auxiliar von Neumann, que é controlado pelo S/CS para a execução de métodos sequenciais quando necessário. Complementarmente, repositórios para *Attributes* e para *Premises/Conditions/Methods* também são definidos, que são regiões de memória responsáveis por armazenar as definições de dados e instruções correspondentes a cada entidade PON respectiva. Também é definido um conjunto de interfaces de E/S mapeadas em memória, utilizadas para permitir o interfaceamento de um processador NOCA com dispositivos externos.

O processador de *Attributes* utiliza o canal de comunicação que o conecta aos processos de *hardware* de *Premises* para enviar uma notificação quando um *Attribute* tem seu valor alterado. Todos os processos de *hardware* responsáveis pelo processamento de *Premises* podem consumir simultaneamente aquela notificação. Contudo, a notificação somente será efetivamente consumida e processada pelo subconjunto de processos aos quais estão alocadas *Premises* cujos cálculos lógico-causais efetivamente dependem daquela notificação. Uma dinâmica similar é implementada para permitir a propagação de notificações, por meio dos respectivos canais de notificação, entre processos de *Premise* e processos de *Condition*, assim como entre os processos de *Condition* e os processos de *Method*.

Adicionalmente, o módulo Escalonador / Resolutor de Conflito (S/CS) também consome as notificações transmitidas por todos os canais de notificação mencionados, o que lhe permite executar os testes e procedimentos de resolução de conflitos. Além disso, o S/CS tem por objetivo verificar se

todos os destinatários de uma notificação estão alocados em processos de *hardware* dos tipos respectivos e alocá-los, caso não estejam, para que possam ser executados e, eventualmente, propaguem as suas próprias notificações. Este mecanismo de alocação e desalocação permite a execução de programas PON com mais entidades do que o número de processos de *hardware* disponíveis, viabilizando desta forma a escalabilidade de aplicações PON conforme proposto nos requisitos.

Em consonância com o modelo dinâmico do PON, cada processo de *hardware* de *Premise* efetua o cálculo lógico correspondente àquela *Premise*, uma vez que receba uma notificação de um *Attribute* do qual dependa. Caso o valor lógico da *Premise* tenha mudado, esta notifica os processos de *hardware* de *Conditions*, os quais consomem esta notificação, caso dependam daquela *Premise*, e efetuam o cálculo lógico correspondente à sua *Condition*. Se o valor lógico desta *Condition* é alterado, esta notifica os processos de *hardware* de *Methods*, os quais consomem esta notificação, caso dependam daquela *Condition*, e efetuam a operação definida pelo *Method*.

Ainda em função dos requisitos propostos, o modelo lógico da NOCA define que os processos de *Method* executem uma operação lógico-aritmética o mais simples possível, a qual tem por objetivo calcular o valor de um *Attribute* e atualizá-lo (ligação indicada por *attribute update* na Fig. 4). Define-se, portanto, dois formatos de operação possíveis:

$$Res = opn1 \text{ } OP \text{ } opn2$$

$$Res = OP \text{ } opn1$$

Ou seja, o processo de *hardware* de *Method* pode executar uma operação *OP* binária sobre operandos *opn1* e *opn2*, gerando um resultado a ser armazenado no *Attribute Res*, ou uma operação *OP* unária sobre o operando *opn1*, gerando um resultado a ser armazenado no *Attribute Res*.

B. Organização arquitetural da NOCA

Com base nas considerações de projeto e no modelo lógico apresentados anteriormente, uma organização arquitetural para a NOCA é proposta conforme se descreve nesta subseção.

Os fundamentos do modelo de execução do PON implicam na potencial propagação em paralelo de notificações entre os elementos da cadeia (*Attribute*, *Premises*, *Conditions*, *Actions*, *Instigations* e *Methods*) e consequente execução do cálculo ou processamento de cada elemento. Portanto, propõe-se que a NOCA seja uma arquitetura de granularidade fina, com processadores reduzidos e dedicados a executar uma instrução da ISA por vez, em função do tipo de elemento PON em cujo processamento aquele processador é especializado:

- PP (*Premise Processor*): responsável por executar o cálculo lógico de uma *Premise*. Ou seja, um PP é a implementação física de um processo de *hardware* de *Premise* conforme proposto no modelo da Fig. 4.

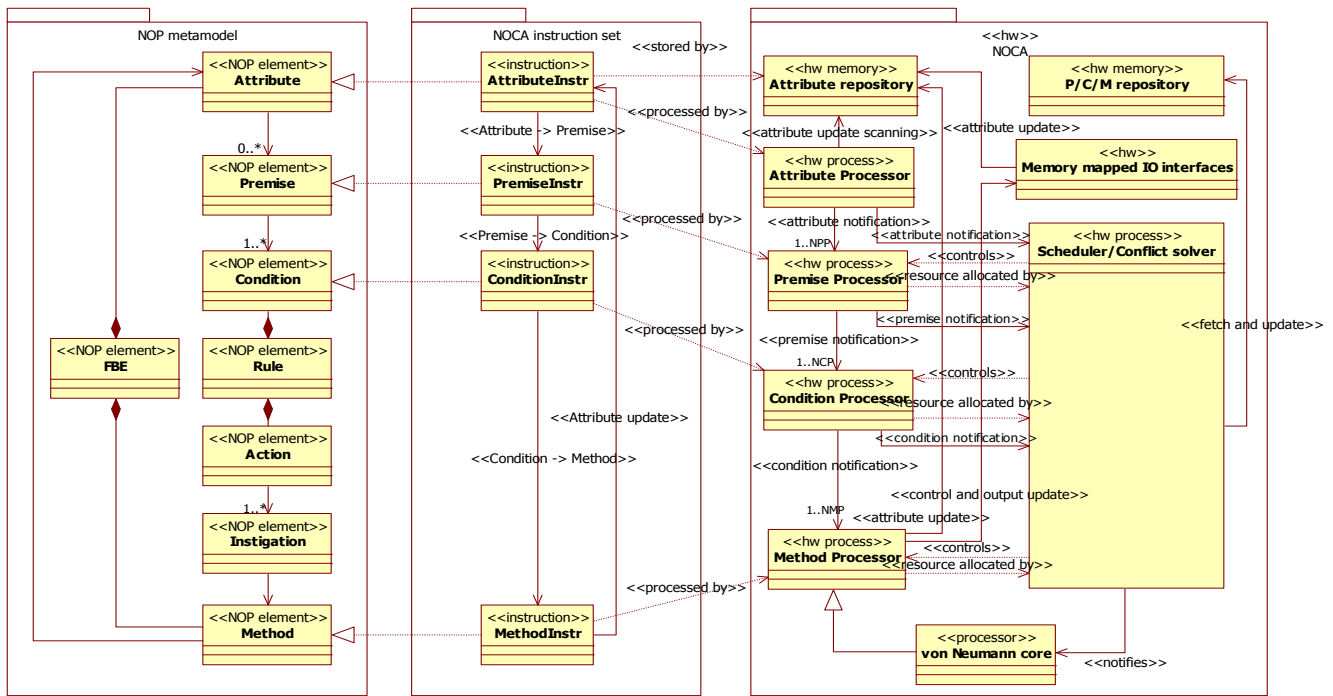


Figura 4. Modelo lógico da NOCA [23].

- CP (*Condition Processor*): responsável por executar o cálculo lógico de uma *Condition*. Ou seja, um CP é a implementação física de um processo de *hardware* de *Condition* conforme proposto no modelo da Fig. 4.
- MP (*Method Processor*): responsável por executar a operação definida por um *Method*. Ou seja, um MP é a implementação física de um processo de *hardware* de *Method* conforme proposto no modelo da Fig. 4.

A Fig. 5 apresenta a organização microarquitetural da NOCA em termos de seus blocos essenciais. Os conjuntos de PPs, CPs e MPs são mostrados ao centro, com as respectivas conexões. O S/CS, por sua vez, é conectado a cada processador de cada conjunto por meio de um barramento de alocação, o que lhe permite efetuar alocações e desalocações de instruções para/de cada processador quando necessário.

De forma a implementar os canais de comunicação apresentados no modelo lógico da Fig. 4, a interface entre cada par de grupos hierárquicos de unidades de processamento (repositório de *Attributes* para PPs, PPs para CPs e CPs para MPs) é efetuado por meio de barramentos/redes de propagação de notificações. Embora este tipo de interconexão possa trazer limitações no que diz respeito a contenção [12][24], na prática esta limitação é minimizada pelo fato de que o tráfego de notificações tende a ser relativamente baixo, dado que estas são enviadas somente quando ocorrem alterações nos valores dos elementos notificantes conforme o modelo do PON.

A parte superior da Fig. 5 apresenta o subsistema de memória, que é dividido em 3 módulos: a Memória de *Attributes* PON, a Memória de *Premises/Conditions/Methods* (P/C/M) PON e a Memória de *startup* e métodos von Neumann. A Memória de *Attributes* PON é responsável por armazenar a base de fatos da aplicação. A Memória de P/C/M, por sua vez, é responsável por armazenar as regras da aplicação, ou seja, o conjunto de *Premises*, *Conditions* e *Methods*. Ambas as memórias podem ser particionadas em dispositivos de memória interna ou externa e, opcionalmente, definir níveis internos de memória *cache* para otimizar o tempo de acesso. Adicionalmente, o subsistema de memória define o bloco de Memória de *startup* e de métodos von Neumann, que é acessada pelo núcleo auxiliar von Neumann para execução das rotinas de inicialização e métodos sequenciais disparados pelo S/CS.

A Fig. 5 também mostra o Ativador de Notificação no lado esquerdo. Este bloco corresponde a um conjunto de periféricos de E/S que podem atualizar valores de *Attributes*, por meio da Rede de Leitura/Escrita de *Attributes*, e desta forma iniciar um novo ciclo de notificação.

C. Conjunto de instruções (ISA) da NOCA

Com base na microarquitetura apresentada na Fig. 5 e em conformidade com o seu modelo lógico, propõe-se o seguinte conjunto de instruções para a NOCA:

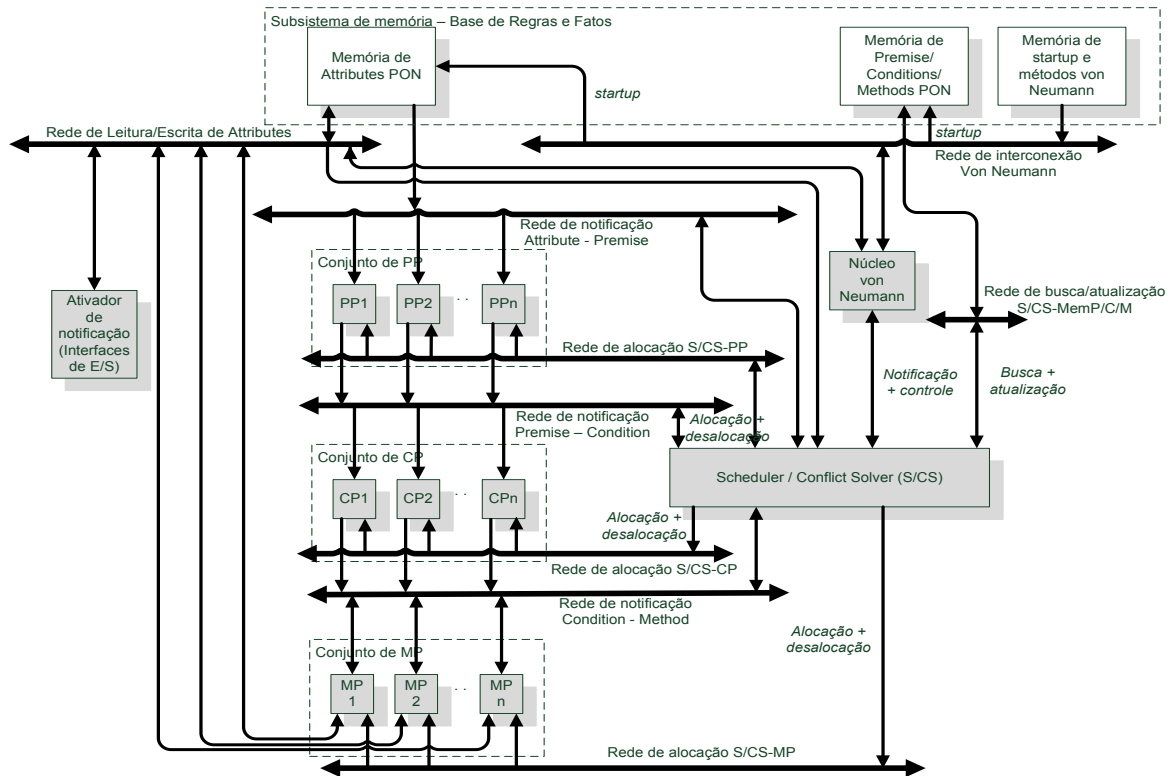


Figura 5. Organização microarquitetural da NOCA [23]

- Instrução ATTRIBUTE-DECL, que define o valor de um *Attribute*, dados de configuração relativos ao seu papel no mecanismo de notificações e uma lista de *Premises* a serem notificadas quando seu valor é alterado. Notar que, embora esta seja uma instrução que define o fluxo de notificação de um *Attribute*, ela também serve como repositório para o valor do dado (variável) correspondente ao estado do *Attribute*, portanto é armazenada na região de memória correspondente ao repositório de *Attributes*.
- Instrução PREMISE-OP, que define ligações para os *Attributes* dos quais depende, a operação relacional a ser efetuada sobre os seus valores pelo PP e uma lista de *Conditions* a serem notificadas quando o seu valor lógico é alterado.
- Instrução CONDITION-OP, que define ligações para as *Premises* de cujos valores lógicos depende, a operação lógica a ser efetuada sobre estes valores pelo CP e uma lista de *Methods* a serem notificados quando o seu valor lógico é alterado.
- Instrução METHOD-OP, que define a ligação para a *Condition* da qual depende, o(s) endereço(s) do(s) operando(s) a ser(em) lido(s) e do *Attribute* de resultado a ser escrito quando é executado (conforme a operação de um *Method* definida no modelo lógico), a operação lógica-aritmética a ser executada pelo MP e o endereço (opcional) de outro *Method* que lhe sucede (método dependente).
- Instrução METHOD-VN-OP, que é executada diretamente pelo escalonador (S/CS) para disparar a execução de um método sequencial pelo núcleo von Neumann auxiliar.

D. Dinâmica de funcionamento da NOCA

Do ponto de vista dinâmico, e em conformidade com o modelo lógico proposto para a NOCA, a execução de uma aplicação PON nesta arquitetura envolve as seguintes operações:

- 1) Execução das rotinas de inicialização (*startup*) pelo núcleo von Neumann, obtidas a partir da Memória de *startup*, com o propósito de inicializar as Memórias de *Attribute* e *P/C/M*. Esta inicialização inclui copiar as instruções de *Attributes* e de *P/C/M* para os respectivos espaços de memória, propagar o valor default dos *Attributes* para as *Premises* dependentes, por meio de notificações, e propagar o valor lógico inicial das *Premises* para as suas *Conditions* dependentes, também por meio de notificações.
- 2) Alteração do valor de algum *Attribute* para início do processo de notificação, o que pode ocorrer tanto ao final do código de *startup* quanto por meio de uma interface de entrada do Ativador de Notificação. Esta alteração gera a propagação de uma notificação para o barramento que interconecta a memória de *Attributes* ao grupo de PPs e ao S/CS; este último tem a tarefa de buscar as instruções PREMISE-OP referenciadas pelo *Attribute* na

memória de *P/C/M* e alocar nos PPs em função de um critério de alocação, caso já não estejam alocadas.

- 3) Cada PP executa a sua instrução *PREMISE-OP* e, eventualmente, gera uma notificação no barramento que interconecta os PPs ao grupo de CPs e ao S/CS; este último tem a tarefa de buscar as instruções *CONDITION-OP* referenciadas pela *Premise* na memória de *P/C/M* e alocar nos CPs em função de um critério de alocação, caso já não estejam alocadas.
- 4) Cada CP executa a sua instrução *CONDITION-OP* e, eventualmente, gera uma notificação no barramento que interconecta os CPs ao grupo de MPs e ao S/CS; este último tem a tarefa de buscar as instruções *METHOD-OP* referenciadas pela *Condition* na memória de *P/C/M* e alocar nos MPs em função de um critério de alocação, caso já não estejam alocadas. Ainda, o S/CS pode disparar uma interrupção para o núcleo von Neumann auxiliar caso a *CONDITION-OP* referencie uma instrução *METHOD-VN-OP*, o que causa a execução de um método sequencial a partir do endereço referenciado por esta instrução.
- 5) Cada MP executa a sua instrução *METHOD-OP* se a *Condition* correspondente notificou valor *true*. A execução de um *METHOD-OP* envolve atualizar o valor de um *Attribute* na memória (via Rede de Leitura/Escrita de *Attributes*), o que pode iniciar um novo ciclo de notificação (passo 2).

V. EXPERIMENTAÇÃO E DISCUSSÃO

A NOCA foi implementada na forma de um primeiro protótipo. Este protótipo foi configurado com 4 PPs, 4 CPs e 4 MPs e implementado em um dispositivo de FPGA Cyclone IV EP4CE115F29C7 com aproximadamente 115000 elementos lógicos. O subsistema de memória foi configurado com espaços de memória *cache* idênticos para *Attributes* e *P/C/M*, cada um composto por 64 blocos com 4 *words* por bloco e modo de escrita *write-through* com associatividade em conjunto de 4 vias. O núcleo von Neumann auxiliar foi implementado com o *soft-core* Altera NiOS [25], sintetizado na mesma FPGA. A frequência do *clock* é de 25 MHz.

Experimentos preliminares foram executados sobre o protótipo da NOCA, conforme apresentado a seguir.

A. Experimento 1 – Sistema de Controle de Tráfego

O primeiro experimento consistiu na comparação de desempenho de uma aplicação, implementada em PON utilizando a ISA da NOCA, com a mesma aplicação implementada em C segundo o PI. Tal aplicação consiste na simulação de um sistema de controle de tráfego utilizando semáforos, que são modelados por meio de entidades que definem as seguintes propriedades: estado atual (“aberto” ou “fechado”), tempo restante até a próxima mudança de estado e número de veículos aguardando aquele semáforo mudar para o estado “aberto”. O número de veículos aguardando é utilizado para ajustar dinamicamente o tempo de fechamento dos semáforos. A quantidade de cruzamentos (e, portanto, de semáforos) pode ser variada a cada novo experimento.

A medida de desempenho obtida consiste no tempo de execução da simulação para o conjunto completo de semáforos, considerando a passagem de *M* unidades de tempo simulado. Em cada uma das *M* unidades de tempo, uma quantidade arbitrária de veículos é inserida na fila de veículos aguardando de cada semáforo no estado “fechado”, eventualmente causando mudanças de estado nos semáforos em função da lógica de simulação.

B. Experimento 2 – Algoritmo de Ordenação

O segundo experimento realizado consistiu na avaliação funcional e de desempenho de uma aplicação que realiza ordenação (*sort*) de elementos numéricos dispostos em um *array*, cujo tamanho pode ser variado para cada novo experimento. Tal aplicação, ao ser concebida segundo o PON, permite explorar o paralelismo de execução de elementos do metamodelo, uma vez que múltiplos pares de elementos adjacentes, do *array* de elementos a serem ordenados, podem ser processados simultaneamente.

Para tanto, o algoritmo define uma cadeia de notificações, como exemplificado na Fig. 6 para a ordenação de 4 elementos. Nesta cadeia, cada instância de elemento do metamodelo do PON é representada por um retângulo, com o respectivo especificador de classe, e os caminhos de notificação são representados por linhas tracejadas direcionadas.

Na cadeia da Fig. 6, cada elemento do *array* é um *Attribute* (*sort_array[0]* a *sort_array[3]*) que notifica a alteração do seu valor, a qual ocorre quando este *Attribute* é trocado (ordenado) com algum *Attribute* adjacente. Para cada par de *Attributes* adjacentes é definida uma *Premise* que verifica se os *Attributes* estão em ordem (*P_esq_>_dir[0]* a *P_esq_>_dir[2]*) e, caso não estejam, notifica uma *Condition* (*C_troca[0]* a *C_troca[2]*) que notifica *Methods* (conjunto de elementos com o especificador de classe *Method*) para efetuar a ordenação daqueles *Attributes*.

O processamento da cadeia pode ser iniciado durante a inserção de cada *Attribute* na posição inicial no *array* (e consequente notificação da *Premise* que verifica a ordem em relação aos adjacentes) ou por meio de um atributo de *trigger* cuja ativação (mudança para o valor 1) é avaliada por uma *Premise* (*P_trig*), que notifica as *Conditions* responsáveis por notificar os *Methods* de troca. Ainda, para cada *Attribute* correspondente a um elemento ordenável, com exceção do primeiro e do último elementos, é definido um *Attribute* que faz o papel de *mutex* (*mutex_array[1]* e *mutex_array[2]*), de tal maneira que *Conditions* de *Rules* de troca adjacentes não acessem simultaneamente o *Attribute* do elemento para troca, o que poderia causar um problema de concorrência.

A medida de desempenho obtida consiste no tempo total de execução da ordenação. O critério de parada é a ordenação de todos os elementos do *array*, o que é verificado externamente à cadeia de notificações por meio de um laço executado pelo núcleo von Neumann. Adicionalmente, o atendimento a este critério se constitui na validação funcional do algoritmo.

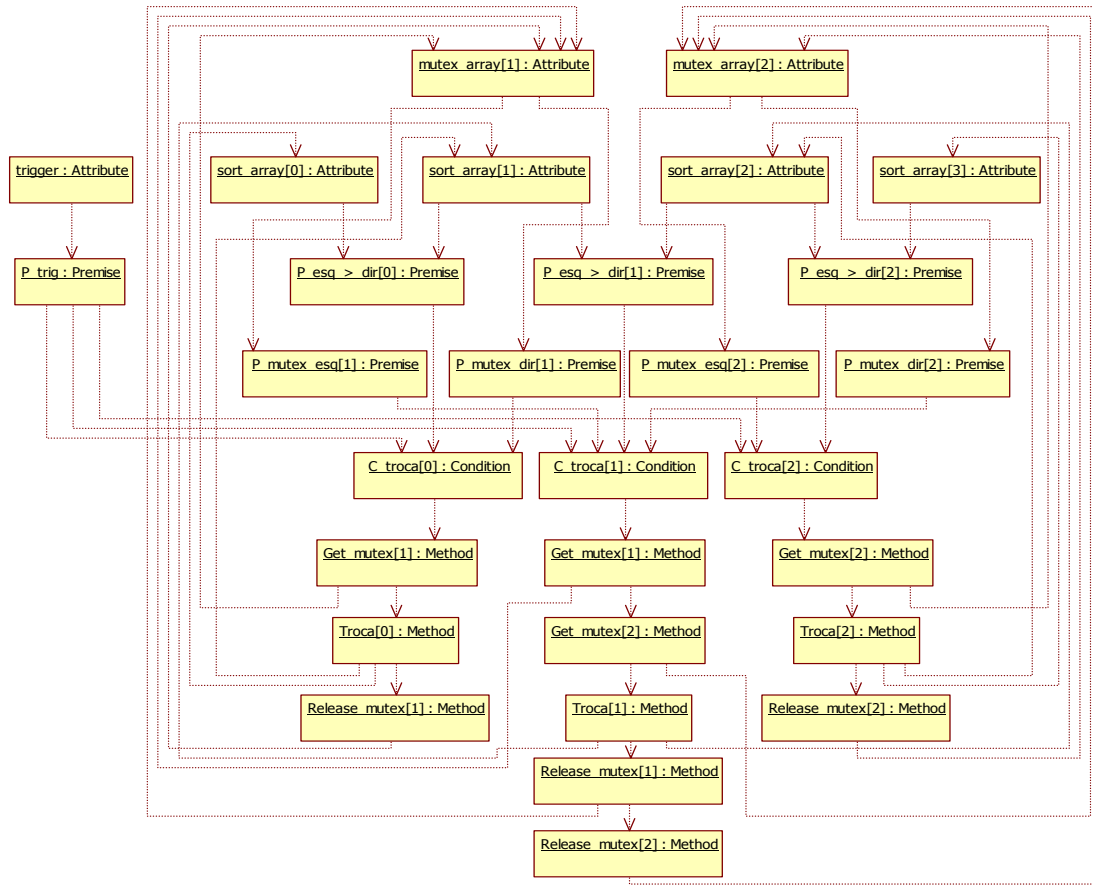


Figura 6. Cadeia de notificações para o experimento do algoritmo de ordenação.

Para fins de análise comparativa, a complexidade assintótica obtida a partir dos dados de desempenho foi comparada à complexidade assintótica obtida a partir de uma implementação específica do mesmo algoritmo sintetizado em FPGA. Esta implementação, diferentemente do protótipo da NOCA, define e sintetiza elementos específicos de *hardware* para cada elemento do metamodelo de notificações, bem como canais de comunicação (notificação) exclusivos e específicos para execução da dinâmica da aplicação de ordenação.

C. Discussão sobre os experimentos

Os dados preliminares obtidos do Experimento 1 mostram que a implementação PON executando em NOCA apresenta melhor desempenho do que a implementação PI, conforme apresentado na Fig. 7, particularmente quando se aumenta o número de semáforos. Isto se deve aos efeitos do “gargalo de von Neumann” na versão PI, que são maiores à medida que a quantidade de dados a serem processados aumenta, ao passo que na implementação PON se verifica que a dinâmica de notificações acaba por minimizar as avaliações com redundância temporal e/ou estrutural.

Em relação ao Experimento 2, os dados preliminarmente obtidos e submetidos a regressão polinomial mostram que o desempenho de ordenação é de complexidade $O(n^2)$, onde n é

o número de elementos a serem ordenados, conforme pode ser verificado na Fig. 8 (a). No entanto, a teoria do cálculo assintótico do PON [26] e também os dados obtidos a partir do experimento comparativo da implementação específica do algoritmo sintetizada em FPGA (Fig. 8 (b)) demonstram que tal algoritmo pode executar com complexidade $O(n)$, dada a paralelização intrínseca do seu funcionamento dinâmico.

Assim, a maior complexidade assintótica da implementação do Experimento 2 executando no protótipo avaliado da NOCA ocorre porque, na prática, a execução do algoritmo de ordenação nesta plataforma é sequencializada devido às operações de alocação e desalocação efetuadas pelo S/CS. Isto decorre do fato de que o protótipo não conta com um número suficiente de unidades de processamento (4 de cada tipo) para que todas as *Premises*, *Conditions* e *Methods* referentes aos possíveis pares de elementos adjacentes que poderiam ser ordenados simultaneamente sejam, de fato, executados em paralelo.

Esta relativamente pequena escala de paralelização, apresentada pelo protótipo da NOCA avaliado, pode ser considerada uma deficiência que afeta o desempenho e a capacidade de execução paralela de ambos os experimentos apresentados.

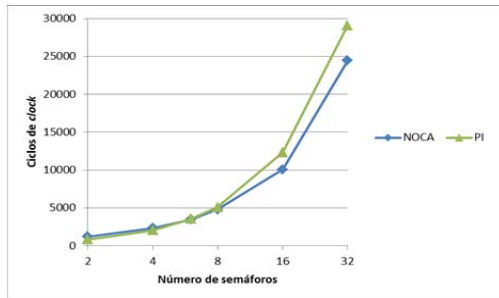


Figura 7. Gráfico dos dados relativos ao Experimento 1.

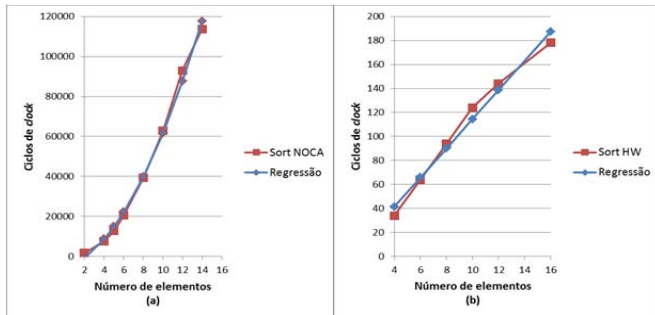


Figura 8. Gráficos dos dados relativos ao Experimento 2.

Outros experimentos preliminares, avaliando uma implementação prototipal da NOCA com 8 processadores de cada tipo, mostraram, no entanto, evidências de que um aumento no número de processadores aumenta proporcionalmente o desempenho de execução. Isto ocorre principalmente porque, diferentemente da dinâmica de um programa von Neumann, as instruções de *Premise*, *Condition* e *Method* são mantidas internamente em cada PP, CP e MP quando alocadas e podem ser reexecutadas (mediante recebimento de notificação) sem necessitar de uma nova busca (*fetch*) na memória, enquanto não forem desalocadas para alocação de uma nova instrução. Assim, quanto mais unidades de processamento houver de cada tipo, menor é a frequência de desalocação e realocação de instruções, o que minimiza os efeitos de latência no acesso à memória de *P/C/M*, dependendo naturalmente das características da aplicação PON sendo executada.

Por outro lado, se a escala de paralelização é baixa, a utilização de um espaço de memória único para armazenar *Premises*, *Conditions* e *Methods* pode se tornar um gargalo devido à relativamente alta utilização do barramento de acesso correspondente pelo S/CS. Embora esta memória implemente um nível de *cache* para melhorar seu desempenho, na prática esta melhoria é minimizada pela falta de localidade espacial de uma aplicação PON, uma vez que o fluxo de execução não é necessariamente sequencial entre instruções em endereços adjacentes de memória.

Dada a granularidade fina proposta para a NOCA e a relativa simplicidade dos PPs, CPs e MPs, em função destes serem processadores dedicados, a tendência é que uma implementação da NOCA de fato seja composta por uma quantidade substancial (dezenas ou centenas) de processadores de cada tipo. Isto tende a favorecer a execução paralela que é intrínseca das aplicações PON e minimizar os efeitos

negativos de latência de acesso à memória mencionados anteriormente.

Em relação à forma como os MPs acessam a memória de *Attributes*, a granularidade fina de execução impede a implementação de um modelo *load/store*, portanto aumentando o número médio de acessos à memória para execução de um *Method*. Embora a baixa quantidade ou mesmo ausência de redundância em *software* PON permita atenuar ou eliminar o *overhead* devido a tais acessos à memória, na prática algumas operações que são tipicamente sequenciais poderiam ser favorecidas por MPs de maior granularidade, a serem implementados em uma versão futura da NOCA. Este MPs poderiam implementar um *pipeline* simples e armazenamento temporário em registradores de acordo com o modelo *load/store*.

VI. CONCLUSÃO E TRABALHOS FUTUROS

A proposta da NOCA é uma alternativa a arquiteturas de computadores paralelos convencionais baseados nos modelos de von Neumann ou de fluxo de dados. Isto se deve ao fato de que a NOCA foi desenvolvida de forma a aderir totalmente a um paradigma emergente chamado PON, cujo modelo de execução é baseado em notificações. Particularmente, isto permite à NOCA se aproveitar da facilidade de execução paralela que é intrínseca à estrutura das aplicações PON.

A NOCA é uma arquitetura flexível e genérica. Flexibilidade e generalidade são obtidas devido ao fato de que, diferentemente de implementações prévias de plataformas de execução para *software* PON, a NOCA permite o desenvolvimento de *software* baseado em um processo tradicional. Isto é, o *software* é buscado de uma memória de programa (portanto, tornando-o editável e flexível) e o *hardware* consiste de uma implementação de processador fixa em termos de arquitetura interna (portanto, genérica para a execução de qualquer aplicação PON). Embora estas características estejam presentes em muitas arquiteturas que são compatíveis com o modelo de von Neumann, é importante enfatizar que a NOCA é fundamentalmente diferente daquelas arquiteturas uma vez que é orientada a notificações (i. e. aderente ao PON).

Quando submetida a experimentos preliminares, NOCA demonstrou melhor desempenho do que uma implementação PI semelhante, com alto grau de otimização, em algumas circunstâncias. Contudo, alguns aspectos foram desvantajosos para o protótipo da NOCA nestes experimentos, em particular a baixa escala de paralelização em função de limitações da plataforma de prototipação e da técnica de implementação utilizada (VHDL em abordagem comportamental). Tais limitações de escala dificultaram, inclusive, a efetiva execução paralela que seria esperada no caso do experimento envolvendo o algoritmo de ordenação, com impacto na complexidade assintótica verificada.

Trabalhos futuros serão focados nos seguintes tópicos: implementação de um simulador da NOCA em *software*, de forma a facilitar a variação da escala de paralelização e a investigação dos seus efeitos na dinâmica de execução; experimentação mais aprofundada do protótipo em *hardware* da NOCA, com aplicações de teste com diferentes

características, e variações na escala de paralelização e avaliação do seu impacto real no desempenho; investigação do efeito da alteração de parâmetros do subsistema de memória da NOCA, particularmente na configuração e na dinâmica da memória *cache*, e do seu efeito no desempenho; e desenvolvimento de técnicas para otimização do código a ser executado na NOCA, inclusive com o desenvolvimento de um compilador adequado.

Como resultado geral desta pesquisa e suas implicações, conclusões mais fundamentadas a respeito das vantagens e deficiências da NOCA (e do PON em si) poderão ser obtidas, permitindo uma melhor avaliação da aplicabilidade desta tecnologia para o desenvolvimento de sistemas em geral.

REFERÊNCIAS

- [1] K. Bousias, L. Guang, C. R. Jesshope and M. Lankamp, "Implementation and evaluation of a microthread architecture". *Journal of Systems Architecture*, 55(3), 2009, 149–161. DOI 10.1016/j.sysarc.2008.07.001.
- [2] S. Borkar and A. A. Chien, "The Future of Microprocessors", in: *Communications of the ACM*, 54(5), p. 67–77, 2011. DOI 10.1145/1941487.
- [3] J. M. Simão and P. C. Stadzisz, "Inference Process Based on Notifications: The Kernel of a Holonic Inference Meta-Model Applied to Control Issues". *IEEE Trans. on Systems, Man and Cybernetics. Part A, Systems and Humans*, V.39, I.1, 238–250, 2009. DOI 10.1109/TSMCA.2008.2006371.
- [4] J. M. Simão and P. C. Stadzisz, "Paradigma Orientado a Notificações (PON) - Uma Técnica de Composição e Execução de Software Orientada a Notificações". Patent pending submitted to INPI/Brazil in 2008 and UTFPR Innovation Agency 2007. INPI Number: PI0805518-1. <http://www.patentesonline.com.br/paradigma-orientado-a-notificacoes-pon-uma-tecnica-de-composicao-e-execucao-de-software-234943.html>.
- [5] J. M. Simão, R. F. Banaszewski, C. A. Tacla and P. C. Stadzisz, "Notification Oriented Paradigm (NOP) and Imperative Paradigm: A Comparative Study", *Journal of Software Engineering and Applications (JSEA)*, Vol. 5, No. 6, 402–416, 2012. DOI 10.4236/jsea.2012.56047.
- [6] E. Peters, R. P. Jasinski, V. A. Pedroni and J. M. Simão, "A New Hardware Coprocessor for Accelerating Notification-Oriented Applications", in: *International Conference on Field-Programmable Technology (FPT)*, 257–260, 2012. DOI 10.1109/FPT.2012.6412145.
- [7] J. M. Simão, R. R. Linhares, F. A. Witt, C. R. E. Lima and P. C. Stadzisz, "Paradigma Orientado a Notificações em Hardware Digital". Patent pending submitted to INPI/Brazil in 2012 and UTFPR Innovation Agency (2012). INPI Provisory Number: BR 10 2012 026429 3.
- [8] J. G. Brookshear, "Computer Science: An Overview". Addison Wesley, 2006.
- [9] M. Gabbrielli and S. Martini, "Programming Languages: Principles and Paradigms". Series: Undergraduate Topics in Computer Science. 1st Edition, 2010.
- [10] J. M. Simão, D. P. B. Renaux, R. R. Linhares and P. C. Stadzisz, "Evaluation of the Notification Oriented Paradigm applied to Sentient Computing". In *Proceedings of the 17th International Symposium of Object/Component-Oriented Real-Time Distributed Computing (ISORC 2014)*, p. 253–260, Reno, USA, June 2014. DOI 10.1109/ISORC.2014.54.
- [11] J. B. Dennis and D. P. Misunas, "A preliminary architecture for a basic data-flow processor", in: *Proceedings of the 2nd annual symposium on Computer architecture (ISCA '75)*. ACM, New York, NY, USA, 126–132, 1974. DOI 10.1145/642089.642111.
- [12] D. A. Patterson and J. L. Hennessy, "Computer Organization & Design: The Hardware / Software Interface". 4th Ed. Morgan Kaufmann Publishers, Inc. San Francisco, CA, USA, 2009.
- [13] T. Ungerer, B. Robic and J. Silc, "A Survey of Processors with Explicit Multithreading", in: *ACM Computing Surveys*, 35(1), 29–63, 2003. DOI 10.1145/641865.641867.
- [14] A. L. S. Gradwohl, "Metalanguage for High-Performance Computing on Hybrid Architectures", in: *IEEE Latin America Transactions*, Vol. 12, No. 6, 1162–1168, 2014.
- [15] P. Kogge, K. Bergman, S. Borkar, D. Campbell, W. Carlson, W. Dally et al., "ExaScale Computing Study: Technology Challenges in Achieving Exascale Systems". September 2008, DARPA. Available in: <http://www.cse.nd.edu/Reports/2008/TR-2008-13.pdf>. Accessed in Feb 7th, 2012.
- [16] J. Backus, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs". *Communications of the ACM*, 21(8), 613–641, 1978. DOI 10.1145/359576.359579.
- [17] S. A. McKee, "Reflections on the Memory Wall", in: *ACM (Eds.), proceedings of the 1st Conference on Computing Frontiers (CF '04)*. New York, USA, 2004. DOI 10.1145/977091.977115.
- [18] T. Ungerer, J. Silc and B. Robic, "Asynchrony in parallel computing: from dataflow to multithreading". In: *Journal of Parallel and Distributed Computing Practices*, 1, 1–33, 1998.
- [19] G. M. Papadopoulos, "Implementation of a General Purpose Dataflow Multiprocessor". Electrical Engineering. MIT Press Cambridge, MA, 1988.
- [20] R. S. Nikhil, K. K. Pingali and Arvind, "I-Structures : Data Structures for Parallel Computing", in: *ACM Transactions on Programming Languages and Systems*, 11(4), 598–632, 1989. DOI 10.1145/69558.69562.
- [21] S. Sakai, Y. Yamaguchi, K. Hiraki, Y. Kodama and T. Yuba, "An Architecture of a Dataflow Single Chip Processor", in: *Proceedings of the 16th annual international symposium on Computer architecture (ISCA '89)*. New York, NY, USA: ACM, 46–53, 1989. DOI 10.1145/74925.74931.
- [22] G. Gupta and G. S. Sohi, "Dataflow Execution of Sequential Imperative Programs on Multicore Architectures", in: *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-44 '11)*. New York, NY, USA: ACM, 59–70, 2011. DOI 10.1145/2155620.2155628.
- [23] R. R. Linhares, P. C. Stadzisz and J. M. Simão, "Arquitetura de Computador Orientada a Notificações - ARQPON". Patent Application submitted to UTFPR Innovation Agency, 2013.
- [24] T. Bjerregaard and S. Mahadevan, "A Survey of Research and Practices of Network-on-Chip", in: *ACM Computing Surveys*, 38(1), 2006. DOI 10.1145/1132952.1132953.
- [25] Altera Corporation, "Altera Nios II Processor Reference", Available in: http://www.altera.com/literature/hb/nios2/n2cpu_nii5v1.pdf, 2011, Accessed in September 27th, 2013.
- [26] J. M. Simão, "A Contribution to the Development of a HMS simulation tool and Proposition of a Meta-Model for Holonic Control", Ph. D. Thesis. Graduate School in Electrical Engineering and Industrial Computer Science (CPGEI) at The Federal Center of Education of Paraná (CEFET-PR) and Research Center For Automatic Control of Nancy (CRAN) - Henry Poincaré University (UHP), 2005.



Robson Ribeiro Linhares was born in Curitiba - PR, Brazil, in 1975. He received in 1999 his B.Sc. in Electrical Engineering, with emphasis on Electronics and Telecommunications, from Federal University of Technology - Paraná (UTFPR), Brazil, which is formerly known as Federal Center for Technological Education of Parana. He received in 2001 his M.Sc. degree in software engineering for real-time systems from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil. He is currently Ph.D. student in computer engineering from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil, as well as Professor with the UTFPR, teaching in a set of educational programs such as the undergraduate courses of Computer Engineering and Information Systems and the Graduate Program in Applied Computing (PPGCA). His research interests include software engineering, programming languages and paradigms, embedded systems and computer architectures.



Jean Marcelo Simão was born in Ponta Grossa - Paraná, Brazil, in 1976. He received in 1994 the Technical degree in informatics from State School of Paraná (CEP) and in 1998 the B.Sc. degree in computer science from State University of Ponta Grossa (UEPG). In 2001, he obtained the M.Sc. degree from Graduate School in Electrical Engineering and Industrial Computer Science (CPGEI) at Federal Center for Technological Education of Parana (CEFET-PR), situated in Curitiba (Brazil),

nowadays named as Federal University of Technology - Paraná (UTFPR). On June 2005, he obtained the Ph.D. degree, after a double thesis, in the domains of industrial computer science at CPGEI/UTFPR and computer engineering & automatics at Research Center for Automatic Control of Nancy (CRAN) - Henry Poincaré University (UHP), situated in France. After, in 2006, he developed teaching activities in a Master at UHP and researches ones at CRAN in a post-doctoral context. Nowadays, he is Associated Professor at UTFPR with affiliation to the Department of Electronics and collaboration with the Departments of Informatics. He is also affiliated to the Laboratory of Innovation in Technology (LIT) of the Centre of Technology Innovation (CITEC) at UTFPR. In 2009, he was affiliated to Graduate School in Applied Computer Science PPGCA from DAINF/UTFPR. Also in 2010, he was affiliated to UTFPR. Finally, his teaching activities concern computer science to engineering courses, whereas his publications and current research interests include agile/holonic manufacturing, artificial intelligence, software/system engineering, computer science, and programming and computer paradigms.



Paulo César Stadzisz was born in Blumenau – SC, Brazil, in 1963. He received in 1987 the Diploma in Data Processing Technologies from the Federal University of Paraná (UFPR), Brazil, and in 1990 the M.Sc. degree in industrial computer science from Graduate Program in Electrical Engineering and Industrial Computer Science (CPGEI) at Federal Center for Technological Education of Paraná (CEFET-PR). He received in 1997 the Ph.D. degree from the Franche-Comté University (France). He is currently Professor at Federal University of Technology - Paraná (UTFPR) teaching in a set of educational programs. He namely teaches and coordinates researches in the CPGEI/UTFPR doctoral program associated to CNPq (National Council for Scientific and Technological Development) and CAPES (Coordination for the Improvement of Higher Education Personnel) of Brazil. He also coordinates the Laboratory of Intelligent Manufacturing Systems (LSIP) and the Laboratory of Innovation in Technology (LIT) at UTFPR. His publications and researches include systems modeling and analysis, simulation, and software engineering.